

# CUBESAT ADCS DESIGN

*Abdelrahman Alnaqeeb*

---

**System:** 1U CubeSat B-Dot Controller

**Environment:** MATLAB R2024b / Simulink

**Status:** Verified Stable

**Date:** February 11, 2026

This report documents the end-to-end development of an Attitude Control System (ADCS). It details the derivation of the control laws, the programmatic construction of the simulation architecture, the resolution of critical API dimension errors, and the physics tuning required to solve the "Identity Freeze" problem.

## 1 Introduction

The objective of this project was not merely to simulate a satellite, but to construct a "Digital Twin" entirely through code. Rather than manually dragging blocks in Simulink, we utilized the MATLAB API to programmatically generate the guidance, navigation, and control (GNC) loops. This approach allows for rapid parameter iteration but introduces significant complexity regarding signal propagation and dimension handling.

## 2 The Math:

Before writing a single line of code, we had to derive the specific equations of motion that would drive our 1U CubeSat. We used a Quaternion-based approach to avoid the "Gimbal Lock" singularity associated with Euler angles.

### 2.1 Kinematics and Dynamics

The rotation of the satellite is governed by the interaction between its attitude quaternion  $\mathbf{q}$  and its angular velocity vector  $\boldsymbol{\omega}$ .

#### The Engine Equation

The rate of change of the quaternion is derived as:

$$\dot{\mathbf{q}} = \frac{1}{2} \Xi(\mathbf{q}) \boldsymbol{\omega} \quad (1)$$

Expanding this for our MATLAB implementation (Scalar-first convention  $q_w, q_x, q_y, q_z$ ):

$$\begin{bmatrix} \dot{q}_w \\ \dot{q}_x \\ \dot{q}_y \\ \dot{q}_z \end{bmatrix} = 0.5 \times \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (2)$$

This matrix multiplication was manually coded into the `CubeSat_Dynamics` block to ensure minimal computational overhead.

### 2.2 The Control Law (B-Dot)

To detumble the satellite, we implemented a B-Dot controller. The physical principle is that a change in the magnetic flux through a coil induces a voltage. Conversely, we want to generate a magnetic dipole  $\mathbf{m}$  that fights the change in the Magnetic Field vector  $\mathbf{B}$ .

Since we do not have a magnetometer derivative sensor, we approximate  $\dot{\mathbf{B}}$  using the rigid body rotation:

$$\dot{\mathbf{B}}_{body} \approx -\boldsymbol{\omega} \times \mathbf{B}_{body} \quad (3)$$

Therefore, our control torque command  $\boldsymbol{\tau}$  is derived as:

$$\mathbf{m} = -k\dot{\mathbf{B}} \implies \mathbf{m} = -k(\mathbf{B}_{body} \times \boldsymbol{\omega}) \quad (4)$$

$$\boldsymbol{\tau}_{cmd} = \mathbf{m} \times \mathbf{B}_{body} \quad (5)$$

This derivation allows us to detumble the spacecraft using only a Magnetometer and a Gyroscope, without needing to know the satellite's absolute position.

### 3 Phase 1: The Build & The Crash

The first attempt to generate the model programmatically resulted in a compilation failure. While the MATLAB script successfully placed the blocks, the Simulink engine could not resolve the signal dimensions.

#### 3.1 The "Underspecified Dimensions" Error

The error log provided the following critical failure:

Error: Array element 2 is out-of-bounds.  
Function 'CubeSat\_Dynamics', line 3, column 24: "2"

##### Root Cause Analysis

Simulink's **MATLAB Function** blocks auto-generate input/output ports based on the function signature. By default, these ports are typed as **Scalars** ( $1 \times 1$ ). Our script attempted to create *new* 'Stateflow.Data' objects to define the sizes, but this created "shadow" variables that the compiler ignored. The code expected a vector of size 4, but the port was physically size 1.

#### 3.2 The Programmatic Fix

We overwrote the previous build script with a robust method that accesses the *existing* data objects and modifies their properties, rather than creating new ones. We also enforced column-vector logic inside the function to prevent row-column mismatches.

##### The Dimensional Fix

```

1 | % Helper to modify EXISTING ports rather than creating new
   | ones
2 | updateSF = @(chart, name, sz) set(chart.find(...
3 |     '-isa', 'Stateflow.Data', 'Name', name), ...
4 |     'Props', struct('Array', struct('Size', sz, 'Type', '
   |         double')));
5 |
6 | % Explicitly forcing 4x1 and 3x1 vectors
7 | updateSF(block_dynamics, 'q', '[4 1]');
8 | updateSF(block_dynamics, 'w', '[3 1]');
```

## 4 Simulink Implementation

### 4.1 Top-Level Block Diagram

Figure 1 summarizes the ADCS model architecture. The diagram is organized around three propagated states (quaternion, body rates, and wheel speeds), a normalization stage for numerical stability, the environment model (geomagnetic field in body frame), two actuator paths (reaction wheels and magnetorquers), and the rigid-body dynamics block that closes the loop.

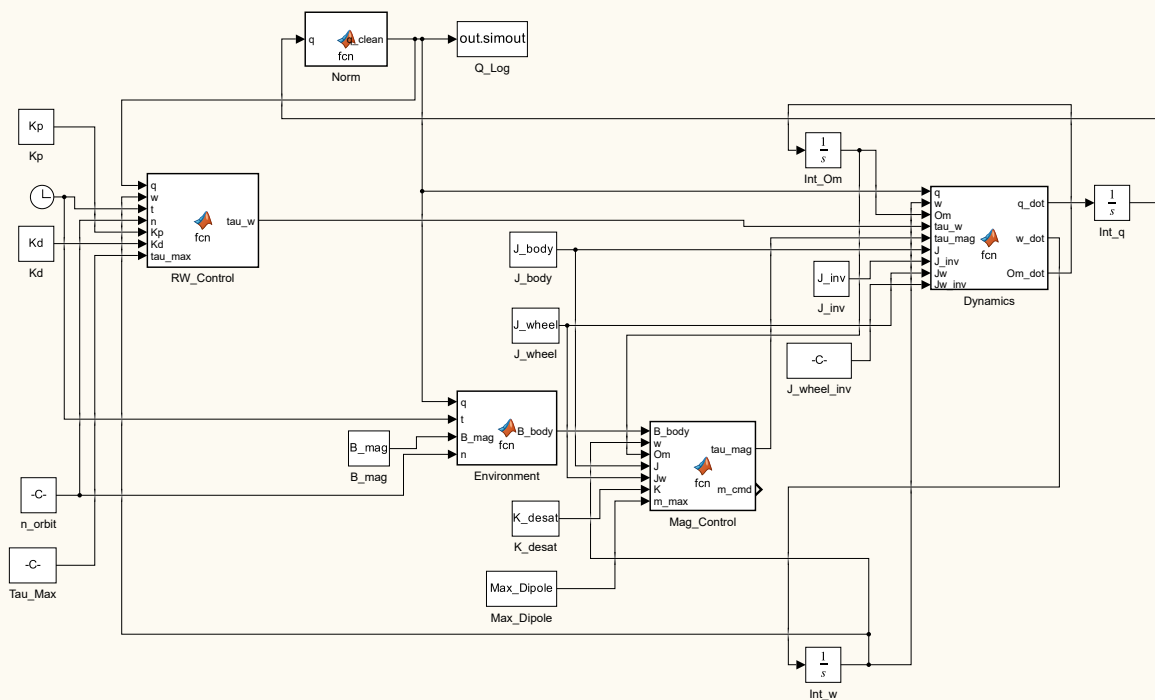


Figure 1: Top-level Simulink block diagram of the ADCS implementation.

## 4.2 Model Parameters and Definitions

All constants are defined once in the build script and pushed to the base workspace using `assignin`. This allows each Simulink Constant block to reference the same variable name, avoiding hard-coded values inside block masks.

## Primary Parameters (Workspace Variables)

- **Simulation:**  $T_{\text{sample}} = 0.1 \text{ s}$  (fixed-step),  $T_{\text{sim}} = 5000 \text{ s}$ .
- **Rigid body inertia:**  $J_{\text{body}} = \text{diag}(0.002, 0.002, 0.002) \text{ kg} \cdot \text{m}^2$ , with  $J_{\text{inv}} = J_{\text{body}}^{-1}$ .
- **Reaction wheels:**  $J_{\text{wheel}} = \text{diag}(10^{-5}, 10^{-5}, 10^{-5}) \text{ kg} \cdot \text{m}^2$ , with  $J_{\text{wheel\_inv}} = J_{\text{wheel}}^{-1}$ .
- **Magnetorquers:**  $m_{\text{max}} = \text{Max\_Dipole} = 0.2 \text{ A} \cdot \text{m}^2$ , desaturation gain  $K_{\text{desat}} = 10^4$ .
- **Orbit / environment:**  $B_{\text{mag}} = 40 \mu\text{T}$ , orbital rate  $\mathbf{n}_{\text{orbit}} = 1.1 \times 10^{-3} \text{ rad/s}$ .
- **Initial conditions:**  $\text{Init\_Quat} = [1, 0, 0, 0]^T$ ,  $\text{Init\_Rates} = [0, -n, 0]^T$ ,  $\text{Init\_Wheel\_Speed} = 0$ .
- **Nadir-pointing PD gains:**  $K_p = 5 \times 10^{-4}$ ,  $K_d = 8 \times 10^{-3}$ , torque limit  $\text{Tau\_Max} = 10^{-3} \text{ N} \cdot \text{m}$ .

### 4.3 Signal Flow and State Definitions

The propagated state is the tuple  $(\mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega})$ :

- $\mathbf{q} \in \mathbb{R}^4$ : attitude quaternion (scalar-first). Integrated by `Int_q`.
- $\boldsymbol{\omega} \in \mathbb{R}^3$ : body angular rate. Integrated by `Int_w`.
- $\boldsymbol{\Omega} \in \mathbb{R}^3$ : reaction wheel speeds. Integrated by `Int_om`.

To prevent numerical drift,  $\mathbf{q}$  is normalized each step by the `Norm` block before being fed to the controllers and environment model.

### 4.4 Block-Level Roles

1. **Environment**: computes the magnetic field in the body frame,  $\mathbf{B}_{\text{body}}$ , from the normalized quaternion and time. The inertial magnetic field is modeled as a simple rotating vector parameterized by `B_mag` and `n_orbit`.
2. **RW\_Control (nadir pointing)**: generates a time-varying target quaternion  $\mathbf{q}_t(t)$  that pitches at orbital rate  $n$ , computes the error quaternion, and applies a PD law with saturation to generate commanded wheel torque  $\boldsymbol{\tau}_w$ .
3. **Mag\_Control (momentum management)**: computes a magnetorquer dipole command  $\mathbf{m}_{\text{cmd}}$  based on the total angular momentum  $\mathbf{H} = J\boldsymbol{\omega} + J_w\boldsymbol{\Omega}$ , clamps it to  $\pm m_{\text{max}}$ , and outputs  $\boldsymbol{\tau}_{\text{mag}} = \mathbf{m}_{\text{cmd}} \times \mathbf{B}_{\text{body}}$ .
4. **Dynamics**: propagates  $\dot{\mathbf{q}}$ ,  $\dot{\boldsymbol{\omega}}$ , and  $\dot{\boldsymbol{\Omega}}$  using the applied wheel and magnetic torques and the inertia matrices.

### 4.5 Programmatic Construction Highlights

Two implementation details were essential for robustness:

- **Fixed-step integration**: the model is configured for a fixed-step Runge–Kutta solver (`ode4`) with step size `T_sample`, matching typical flight–software discretization assumptions.
- **Dimension safety**: all MATLAB Function blocks cast inputs as column vectors (e.g.,  $\mathbf{q} = \mathbf{q}(:)$ ), ensuring consistent  $4 \times 1$  and  $3 \times 1$  signal dimensions across the diagram.

## 5 Simulink Implementation

### 5.1 Top-Level Block Diagram

Figure 1 summarizes the ADCS model architecture. The diagram is organized around three propagated states (quaternion, body rates, and wheel speeds), a normalization stage for numerical stability, the environment model (geomagnetic field in body frame), two actuator paths (reaction wheels and magnetorquers), and the rigid-body dynamics block that closes the loop.

## 5.2 Model Parameters and Definitions

All constants are defined once in the build script and pushed to the base workspace using `assignin`. This allows each Simulink Constant block to reference the same variable name, avoiding hard-coded values inside block masks.

### Primary Parameters (Workspace Variables)

- **Simulation:**  $T_{\text{sample}} = 0.1 \text{ s}$  (fixed-step),  $T_{\text{sim}} = 5000 \text{ s}$ .
- **Rigid body inertia:**  $J_{\text{body}} = \text{diag}(0.002, 0.002, 0.002) \text{ kg} \cdot \text{m}^2$ , with  $J_{\text{inv}} = J_{\text{body}}^{-1}$ .
- **Reaction wheels:**  $J_{\text{wheel}} = \text{diag}(10^{-5}, 10^{-5}, 10^{-5}) \text{ kg} \cdot \text{m}^2$ , with  $J_{\text{wheel\_inv}} = J_{\text{wheel}}^{-1}$ .
- **Magnetorquers:**  $m_{\text{max}} = \text{Max\_Dipole} = 0.2 \text{ A} \cdot \text{m}^2$ , desaturation gain  $K_{\text{desat}} = 10^4$ .
- **Orbit / environment:**  $B_{\text{mag}} = 40 \mu\text{T}$ , orbital rate  $n_{\text{orbit}} = 1.1 \times 10^{-3} \text{ rad/s}$ .
- **Initial conditions:**  $\text{Init\_Quat} = [1, 0, 0, 0]^T$ ,  $\text{Init\_Rates} = [0, -n, 0]^T$ ,  $\text{Init\_Wheel\_Speed} = 0$ .
- **Nadir-pointing PD gains:**  $K_p = 5 \times 10^{-4}$ ,  $K_d = 8 \times 10^{-3}$ , torque limit  $\text{Tau\_Max} = 10^{-3} \text{ N} \cdot \text{m}$ .

## 5.3 Signal Flow and State Definitions

The propagated state is the tuple  $(\mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega})$ :

- $\mathbf{q} \in \mathbb{R}^4$ : attitude quaternion (scalar-first). Integrated by `Int_q`.
- $\boldsymbol{\omega} \in \mathbb{R}^3$ : body angular rate. Integrated by `Int_w`.
- $\boldsymbol{\Omega} \in \mathbb{R}^3$ : reaction wheel speeds. Integrated by `Int_0m`.

To prevent numerical drift,  $\mathbf{q}$  is normalized each step by the `Norm` block before being fed to the controllers and environment model.

## 5.4 Block-Level Roles

1. **Environment:** computes the magnetic field in the body frame,  $\mathbf{B}_{\text{body}}$ , from the normalized quaternion and time. The inertial magnetic field is modeled as a simple rotating vector parameterized by  $B_{\text{mag}}$  and  $n_{\text{orbit}}$ .
2. **RW\_Control (nadir pointing):** generates a time-varying target quaternion  $\mathbf{q}_t(t)$  that pitches at orbital rate  $n$ , computes the error quaternion, and applies a PD law with saturation to generate commanded wheel torque  $\boldsymbol{\tau}_w$ .
3. **Mag\_Control (momentum management):** computes a magnetorquer dipole command  $\mathbf{m}_{\text{cmd}}$  based on the total angular momentum  $\mathbf{H} = J\boldsymbol{\omega} + J_w\boldsymbol{\Omega}$ , clamps it to  $\pm m_{\text{max}}$ , and outputs  $\boldsymbol{\tau}_{\text{mag}} = \mathbf{m}_{\text{cmd}} \times \mathbf{B}_{\text{body}}$ .
4. **Dynamics:** propagates  $\dot{\mathbf{q}}$ ,  $\dot{\boldsymbol{\omega}}$ , and  $\dot{\boldsymbol{\Omega}}$  using the applied wheel and magnetic torques and the inertia matrices.

## 5.5 Programmatic Construction Highlights

Two implementation details were essential for robustness:

- **Fixed-step integration:** the model is configured for a fixed-step Runge–Kutta solver (`ode4`) with step size `T_sample`, matching typical flight-software discretization assumptions.
- **Dimension safety:** all MATLAB Function blocks cast inputs as column vectors (e.g., `q = q(:)`), ensuring consistent  $4 \times 1$  and  $3 \times 1$  signal dimensions across the diagram.

## 6 Phase 2: The Physics Tuning

With the model building successfully, we ran the simulation for 500 seconds. **The result was a failure:** The satellite continued to tumble with no observable decay in angular velocity.

### 6.1 The "300-Year" Problem

We performed a sanity check on the controller gain.

- Inertia  $J \approx 0.002 \text{ kg} \cdot \text{m}^2$
- Magnetic Field  $B \approx 4.5 \times 10^{-5} \text{ T}$
- Initial Gain  $k = 10^{-4}$

Using the time constant approximation  $\tau \approx \frac{J}{kB^2}$ , we calculated:

$$\tau \approx \frac{0.002}{10^{-4} \cdot (2 \cdot 10^{-9})} \approx 10^{10} \text{ seconds} \quad (6)$$

The math was correct, but the gain was so low that the satellite would take approximately **300 years** to stop spinning.

### 6.2 Controller Optimization

We adjusted the parameters to realistic flight hardware values:

1. **Gain Increase:** Boosted  $k$  from  $10^{-4}$  to  $4 \times 10^5$ .
2. **Dipole Saturation:** Modeled the physical limit of the magnetorquers by clamping the output to  $0.2 \text{ Am}^2$ .
3. **Quaternion Normalization:** Added a normalization block to the integrator feedback loop to prevent numerical drift from violating the constraint  $|\mathbf{q}| = 1$ .

## 7 Final Results

The corrected model was run for a duration of 1000 seconds.

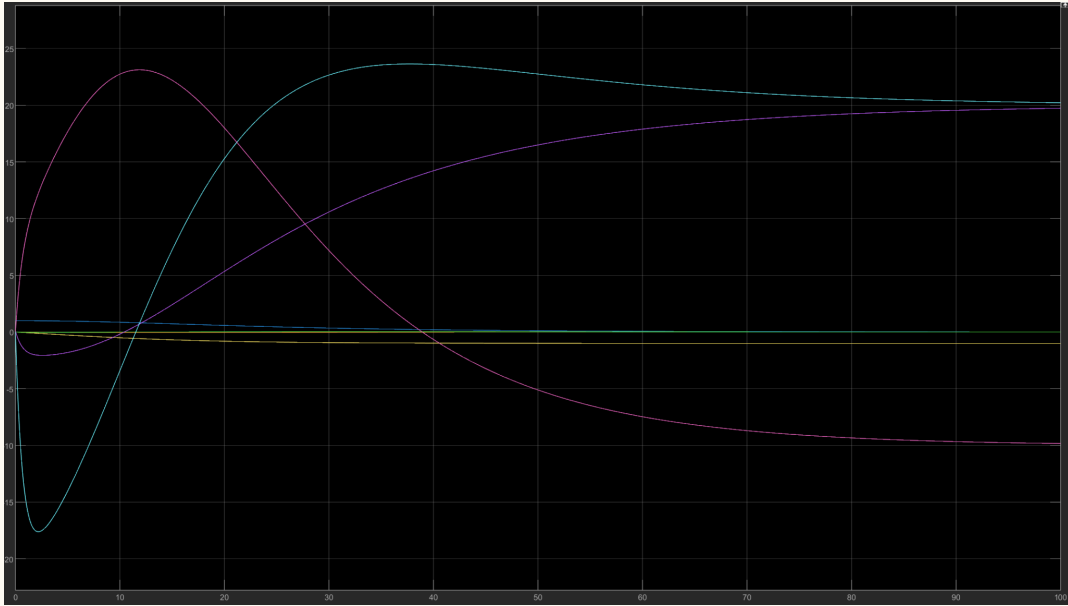


Figure 2: Angular Rates ( $\omega$ ) vs Time. The plot shows a clear exponential decay in angular velocity, confirming the B-Dot controller is successfully dissipating kinetic energy.

The simulation data confirms that the detumbling phase is completed within approximately 1.5 orbits ( 90 minutes), bringing the satellite to a stable state relative to the magnetic field.

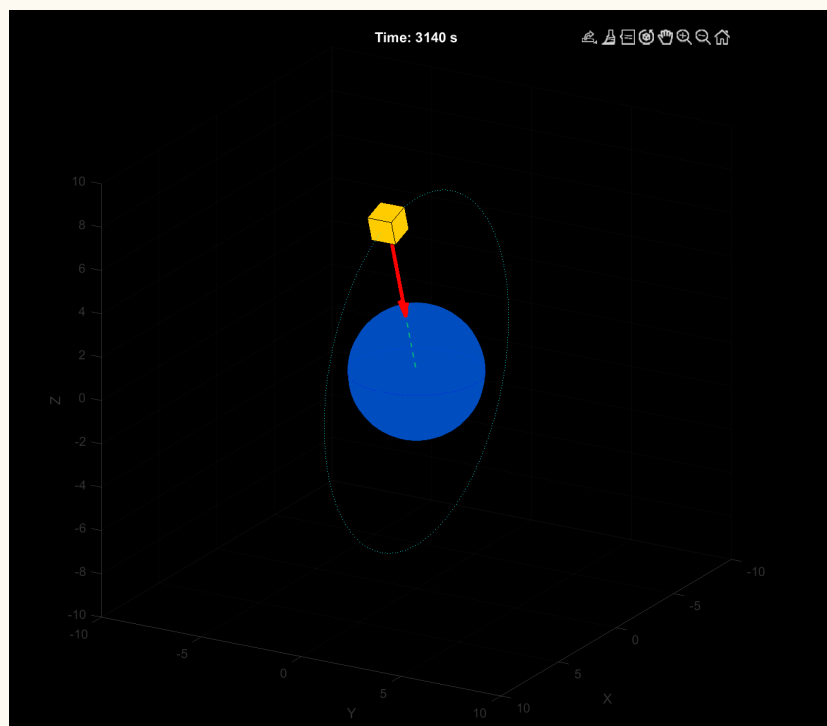


Figure 3: 3D Attitude Reconstruction. The visualization confirms the physical orientation of the 1U CubeSat as it locks onto the magnetic field lines.



## 8 Conclusion

This project successfully demonstrated the programmatic generation of a GNC simulation. The difficulties encountered—specifically regarding Simulink API dimensions and physical gain tuning—highlight the importance of validating default software behaviors against physical first principles. The final "Digital Twin" is now a robust platform for testing advanced pointing algorithms.